# 8 优化内存访问

现代计算机仍然基于经典的冯 · 诺伊曼体系结构构建，其中包括 CPU、内存和输入/输出单元。内存操作（加载和存储）占据了性能瓶颈和功耗的最大部分。毫无疑问，我们首先从这个类别开始。

关于内存层次结构性能非常重要的说法得到了图 62 的支持。它显示了内存和处理器之间性能差距的增长。垂直轴是对数刻度，显示了 CPU-DRAM 性能差距的增长。内存基线是来自 1980 年的 64 KB DRAM 芯片的内存访问延迟。典型的 DRAM 性能改进为每年 7%，而 CPU 每年享受 20-50% 的改进。[Hennessy & Patterson, 2017]

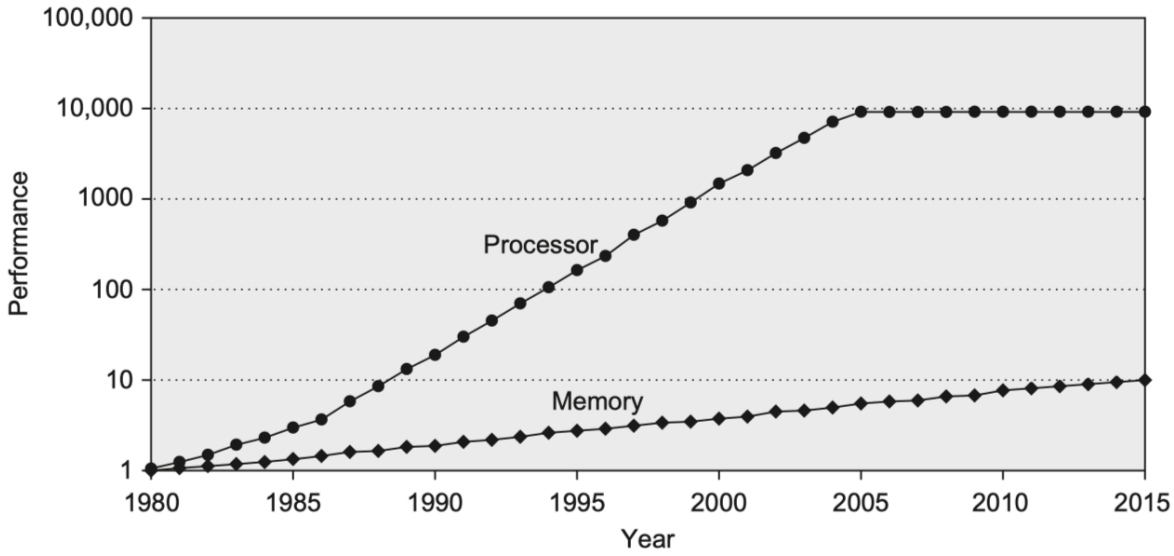



Figure 62: 内存和处理器之间性能差距。© 图片来自 [Hennessy & Patterson, 2017]。

确实，一个变量可以在最小的 L1 缓存中在几个时钟周期内获取，但如果不在 CPU 缓存中，则从 DRAM 获取该变量可能需要超过三百个时钟周期。从 CPU 的角度来看，最后一级缓存未命中感觉就像是一个非常长的时间，特别是如果处理器在此期间没有执行任何有用的工作。当系统高度加载线程，并且没有可用的内存带宽及时满足所有加载和存储时，执行线程也可能会饿死。

当应用程序执行大量内存访问并花费大量时间等待它们完成时，这样的应用程序被描述为受内存限制。这意味着为了进一步提高其性能，我们可能需要改进如何访问内存，减少此类访问的数量或升级内存子系统本身。

在 TMA 方法论中，`Memory Bound` 估算了由于对加载或存储指令的需求而导致 CPU 流水线可能停滞的插槽的比例。解决这样的性能问题的第一步是找到导致高 `Memory Bound` 指标的内存访问（参见 Section 6.1.1）。一旦确定了有问题的内存访问，就可以应用几种优化策略。下面我们将讨论几种典型情况。

## 8.1 缓存友好数据结构

编写缓存友好的算法和数据结构是打造高性能应用程序的重要因素之一。缓存友好代码的关键支柱是我们之前在 Section 3.6 描述的局部性原理，包括时间局部性和空间局部性。这里的目标是允许高效地从缓存中获取所需数据。在设计缓存友好代码时，考虑缓存行而不只是单个变量及其在内存中的位置会很有帮助。

### 8.1.1 顺序访问数据

利用缓存空间局部性的最佳方式是进行顺序内存访问。通过这样做，我们可以让硬件预取器（参见 Section 3.6.1.5.1）识别内存访问模式并提前引入下一块数据。Listing 134 中展示了 C 代码示例，它执行了此类缓存友好访问。该代码之所以“缓存友好”，是因为它按照矩阵在内存中的布局顺序访问矩阵元素（行优先遍历: https://en.wikipedia.org/wiki/Row-_and_column-major_order[134]）。交换数组中索引的顺序（即 `matrix[column][row]`）将导致以列为主的矩阵遍历，这不会利用空间局部性并会损害性能。

代码清单: 缓存友好的内存访问。

```
for (row = 0; row < NUMROWS; row++)
  for (column = 0; column < NUMCOLUMNS; column++)
    matrix[row][column] = row + column;
```

Listing 134 中展示的例子是一个经典例子，但现实世界中的应用程序通常要复杂得多。有时您需要付出更多努力才能编写缓存友好的代码。例如，在已排序的大数组中进行二分搜索的标准实现并没有利用空间局部性，因为它测试的是彼此相距甚远、不共享同一缓存行的不同位置的元素。解决这个问题最著名的方式是使用 Eytzinger 布局存储数组元素 [Khuong & Morin, 2015]。它的想法是使用类似 BFS 的布局（通常在二进制堆中看到）将一个隐式的二叉搜索树打包到数组中。如果代码在数组中执行大量二分搜索，将其转换为 Eytzinger 布局可能会有益处。

### 8.1.2 使用合适的容器

几乎任何语言都提供各种现成的容器。但了解它们的底层存储和性能影响很重要。[Fog, 2004, Chapter 9.7 Data structures, and container classes] 提供了一个很好的关于选择合适的 C++ 容器的逐步指南。

此外，选择数据存储时要考虑代码将如何使用它。考虑一种情况，需要在数组中存储对象与存储指向这些对象的指针之间进行选择，而对象的大小较大。指针数组占用更少的内存。这将使修改数组的操作受益，因为指针数组需要更少的内存传输。然而，当保留对象本身时，通过数组进行线性扫描会更快，因为它更符合缓存，不需要间接内存访问。[135]

### 8.1.3 数据压缩

通过使数据更紧凑可以改善内存层次结构的利用率。有许多方法可以压缩数据。经典示例之一是使用位字段。Listing 8.1.3 展示了在数据打包时代码可能获益的例子。如果我们知道 `a`, `b` 和 `c` 代表需要一定数量位才能编码的枚举值，我们就可以减少结构体 `S` 的存储空间（参见 Listing 8.1.3）。

代码清单: 打包数据: 基线结构体。

```
struct S {
  unsigned a;
  unsigned b;
  unsigned c;
}; // S is `sizeof(unsigned int) * 3` bytes
```

代码清单: 打包数据: 打包的结构体。

```
struct S {
  unsigned a:4;
  unsigned b:2;
```

[134] 行优先和列优先顺序 - https://en.wikipedia.org/wiki/Row-_and_column-major_order。

[135] 博客文章“对象向量 vs 指针向量”作者 B. Filipek - https://www.bfilipek.com/2014/05/vector-of-objects-vs-vector-of-pointers.html。

```
  unsigned c:2;
}; // S is only 1 byte
```

这大大减少了来回传输的内存量并节省了缓存空间。请记住，这会带来访问每个打包元素的成本。由于 b 的位与 a 和 c 共享同一个机器字，编译器需要执行 >> (右移) 和 & (AND) 操作来加载它。类似地，需要 << (左移) 和 | (OR) 操作将值存储回去。在额外计算比低效内存传输引起的延迟更便宜的地方，数据打包是有益的。

此外，当避免编译器添加的填充（参见 Listing 8.1.3 中的示例）时，程序员可以通过重新排列结构或类中的字段来减少内存使用。编译器插入未使用的内存字节（填充）的原因是为了允许高效地存储和获取结构的单个成员。在该示例中，如果将 S1 的成员按其大小递减的顺序声明，则可以减小其大小。

代码清单: 避免编译填充。

```
struct S1 {
  bool b;
  int i;
  short s;
}; // S1 is `sizeof(int) * 3` bytes

struct S2 {
  int i;
  short s;
  bool b;
}; // S2 is `sizeof(int) * 2` bytes
```

### 8.1.4 对齐和填充

改善内存子系统利用率的另一个技术是对齐数据。可能出现这种情况，即一个大小为 16 字节的对象占用两个缓存行，即它从一个缓存行开始并结束于下一个缓存行。获取这样一个对象需要两个缓存行读取，如果对象正确对齐，可以避免这种情况。Listing 8.1.4 展示了如何使用 C++11 的 alignas 关键字对齐内存对象。

代码清单: 使用 “alignas” 关键字对齐数据。

```
// Make an aligned array
alignas(16) int16_t a[N];

// Objects of struct S are aligned at cache line boundaries
#define CACHELINE_ALIGN alignas(64)
struct CACHELINE_ALIGN S {
  //...
};
```

如果变量存储在可被其自身大小整除的内存地址上，则访问它的效率最高。例如，一个 double 类型变量占用 8 个字节的存储空间，因此最好将其存储在一个可被 8 整除的地址上。这个大小通常是 2 的幂次方。大于 16 个字节的对象应该存储在一个可被 16 整除的地址上。[Fog, 2004]

对齐可能会导致未使用字节的空洞，从而降低内存带宽利用率。在上面的例子中，如果结构体 S 只有 40 个字节，那么下一个 S 对象将会从下一个缓存行的开头开始，这会在每个保存 S 结构体的缓存行中留下 64 - 40 = 24 个未使用的字节。

有时需要填充数据结构成员以避免一些极端情况，例如缓存竞争 [Fog, 2004, Chapter 9.10 Cache contentions] 和伪共享 (参见 Section 13.7.3)。例如，在多线程应用程序中，当两个线程 A 和 B 访问同一结构的不同字段时，可能会出现伪共享问题。Listing 8.1.4 展示了可能发生这种情况的代码示例。由于结构体 S 的成员 a 和 b 可能占据同一个缓存行，因此缓存一致性问题可能会显著减慢程序运行速度。为了解决这个问题，可以填充 S 使得成员 a 和 b 不共享同一个缓存行，如 Listing 8.1.4 所示。

代码清单: 填充数据: 基线版本。

```
struct S {
  int a; // written by thread A
  int b; // written by thread B
};
```

代码清单: 填充数据: 改进版本。

```
#define CACHELINE_ALIGN alignas(64)
struct S {
  int a; // written by thread A
  CACHELINE_ALIGN int b; // written by thread B
};
```

使用 malloc 进行动态分配时，保证返回的内存地址满足目标平台的最小对齐要求。一些应用程序可能受益于更严格的对齐。例如，以 64 字节对齐而不是默认的 16 字节对齐动态分配 16 字节。POSIX 系统的用户可以利用 memalign: https://linux.die.net/man/3/memalign[136] API 来实现这一目的。其他人可以像 这里: https://embeddedartistry.com/blog/2017/02/22/generating-aligned-memory/[137] 所描述的那样自己实现。

对齐考虑最重要的领域之一是 SIMD 代码。当依赖于编译器自动向量化时，开发人员无需做任何特殊操作。但是，当您使用编译器向量内联函数编写代码时 (参见 Section 9.4)，它们通常要求地址可被 16、32 或 64 整除。编译器内联头文件中提供的向量类型已经做了注释，以确保适当的对齐。[Fog, 2004]

```
// ptr will be aligned by alignof(__m512) if using C++17
__m512 * ptr = new __m512[N];
```

### 8.1.5 动态内存分配

首先，有很多可以替代 malloc 的工具，它们更快、更可扩展，并且更好地解决了碎片化[138]问题。仅仅通过使用非标准内存分配器，你就可以获得 2% 左右的性能提升。动态内存分配的一个典型问题是，在启动时，线程会争相尝试同时分配内存区域。[139] 最受欢迎的内存分配库之一是 jemalloc: http://jemalloc.net/[140] 和 tcmalloc: https://github.com/google/tcmalloc[141]。

其次，可以使用自定义分配器来加速分配，例如 arena 分配器: https://en.wikipedia.org/wiki/Region-based_memory_management[142]。它们的主要优势之一是开销低，因为此类分配器不会为每次内存分配执行系统调用。另一个优点是它的高灵活性。开发人员可以根据操作系统提供的内存区域实现自己的分配策略。一个简单的方法是维护两个不

[136] Linux 手册页面，用于 memalign - https://linux.die.net/man/3/memalign。
[137] 生成对齐内存- https://embeddedartistry.com/blog/2017/02/22/generating-aligned-memory/。典型的malloc实现涉及同步，以防止多个线程试图动态分配内存
[138] 碎片化- https://en.wikipedia.org/wiki/Fragmentation_(computing)。
[139] 相同的情况也适用于内存释放。
[140] jemalloc - [http://jemalloc.net/] (http://jemalloc.net/)。
[141] tcmalloc - https://github.com/google/tcmalloc
[142] 基于区域的内存管理- https://en.wikipedia.org/wiki/Region-based_memory_management

同的分配器，每个分配器都有自己的 arena（内存区域）：一个用于热门数据，另一个用于冷数据。将热门数据放在一起可以使其共享缓存行，从而提高内存带宽利用率和空间局部性。它也改善了 TLB 利用率，因为热门数据占用的内存页面更少。此外，自定义内存分配器可以使用线程局部存储来实现每个线程的分配，并摆脱线程之间的任何同步。当应用程序基于线程池并且不会产生大量线程时，这变得很有用。

### 8.1.6 调整代码以适应内存层次结构

某些应用程序的性能取决于特定级别缓存的大小。最著名的例子是通过循环嵌套优化（平铺 (tiling)）来改进矩阵乘法。这个想法是将矩阵的工作大小分解成更小的部分（平铺块 (tiles)），以便每个平铺块都能适应 L2 缓存[143]。大多数架构提供了类似CPUID的指令[144]，这允许我们查询缓存的大小。或者，可以使用缓存无关算法[145]，其目标是在任何大小的缓存下都能合理地工作。

Intel CPU 具有数据线性地址硬件特性（参见 Section 6.6），这支持在 easyperf博客文章[146]中描述的缓存块 (cache blocking)。附：intel Cache Blocking Techniques

## 8.2 显式内存预取

到现在为止，您应该已经知道未能在缓存中解析的内存访问通常代价高昂。现代 CPU 非常努力地降低预取请求提前足够发出时的缓存未命中惩罚。如果请求的内存位置不在缓存中，我们将无论如何遭受缓存未命中，因为我们必须访问 DRAM 并提取数据。但是，如果我们在程序需要数据时将该内存位置引入缓存，那么我们实际上可以将缓存未命中惩罚降为零。

现代 CPU 有两种解决这个问题的机制：硬件预取和 OOO 执行。硬件预取器通过对重复内存访问模式发起预取请求来帮助隐藏内存访问延迟。而 OOO 引擎则向前看 N 条指令，并提前发出加载指令，以允许平滑执行未来将需要此数据的指令。

当数据访问模式太复杂无法预测时，硬件预取器就会失败。软件开发人员对此无能为力，因为我们无法控制该单元的行为。另一方面，OOO 引擎不像硬件预取器那样试图预测未来需要的内存位置。因此，它成功的唯一衡量标准是它通过提前调度加载隐藏了多少延迟。

考虑 Listing 8.2 中的一段代码片段，其中 `arr` 是一个包含一百万个整数的数组。索引 `idx` 被赋予一个随机值，然后立即用于访问 `arr` 中的一个位置，该位置几乎肯定会错过缓存，因为它是随机的。硬件预取器不可能预测，因为每次加载都进入内存中一个完全新的位置。从知道内存位置的地址（从函数 `random_distribution` 返回）到需要该内存位置的值（调用 `doSomeExtensiveComputation`）的时间间隔称为 预取窗口。在这个例子中，由于预取窗口非常小，OOO 引擎没有机会提前发出加载指令。这导致内存访问 `arr[idx]` 的延迟成为执行循环时的关键路径，如图 63 所示。可以看到，程序在没有取得值的情况下等待（阴影填充矩形），无法向前推进。

代码清单: 随机数为后续加载提供数据。

```
for (int i = 0; i < N; ++i) {
  size_t idx = random_distribution(generator);
  int x = arr[idx]; // cache miss
  doSomeExtensiveComputation(x);
}
```

[143] 通常情况下，人们会针对 L2 缓存的大小进行调整，因为它是内核之间不共享的。

[144] 英特尔处理器的 CPUID 指令在 [Intel, 2023b, Volume 2] 中描述。

[145] 缓存无关算法- https://en.wikipedia.org/wiki/Cache-oblivious_algorithm。

[146] 博客文章“检测伪共享”- https://easyperf.net/blog/2019/12/17/Detecting-false-sharing-using-perf#2-tune-the-code-for-better-utilization-of-cache-hierarchy。

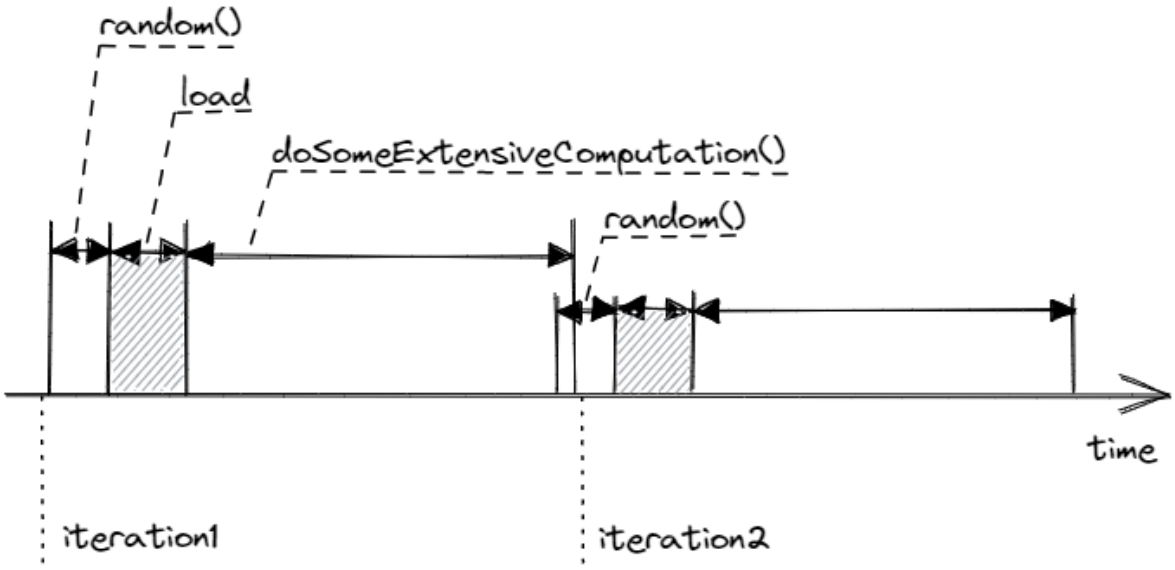



Figure 63: 显示关键路径上的负载延迟的执行时间线.

这里还有另一个重要观察。当 CPU 接近完成第一次迭代时，它会推测性地开始执行来自第二次迭代的指令。这在迭代之间创建了一个积极的执行重叠。然而，即使在现代处理器中，也缺少足够的 OOO 功能，无法完全将缓存未命中延迟与来自迭代 1 的 `doSomeExtensiveComputation` 的执行重叠。换句话说，在我们的例子中，CPU 无法提前查看当前执行，以便足够早地发出加载指令。

幸运的是，这并不是死路一条，因为有一种方法可以加速这段代码。为了隐藏缓存未命中延迟，我们需要将其与 `doSomeExtensiveComputation` 的执行重叠。如果我们流水线化随机数生成并在下一次迭代中开始预取内存位置，就可以实现这一点，如 Listing 147 所示。请注意使用 `__builtin_prefetch`: https://gcc.gnu.org/onlinedocs/gcc/Other-Builtins.html,[147] 开发人员可以使用的特殊提示，明确请求 CPU 预取特定内存位置。图 64 展示了这种转换的图形说明。

代码清单: 利用明确的软件内存预取提示。

```
size_t idx = random_distribution(generator);
for (int i = 0; i < N; ++i) {
  int x = arr[idx];
  idx = random_distribution(generator);
  // prefetch the element for the next iteration
  __builtin_prefetch(&arr[idx]);
  doSomeExtensiveComputation(x);
}
```

在 x86 平台上利用显式软件预取的另一种选择是使用编译器内部函数 `_mm_prefetch`。有关更多详细信息，请参见 Intel 内部函数指南。无论如何，编译器都会将其编译成机器指令：x86 的 `PREFETCH` 和 ARM 的 `pld`。对于某些平台，编译器可能会跳过插入指令，因此检查生成的机器代码是一个好主意。

存在一些情况下，软件内存预取是不可能的。例如，当遍历链表时，预取窗口非常小，无法隐藏指针追逐的延迟。

在 Listing 147 中，我们看到了针对下一次迭代进行预取的示例，但您也经常会遇到需要为 2、4、8 甚至更多次迭代进行

[147] GCC 内置程序- https://gcc.gnu.org/onlinedocs/gcc/Other-Builtins.html。

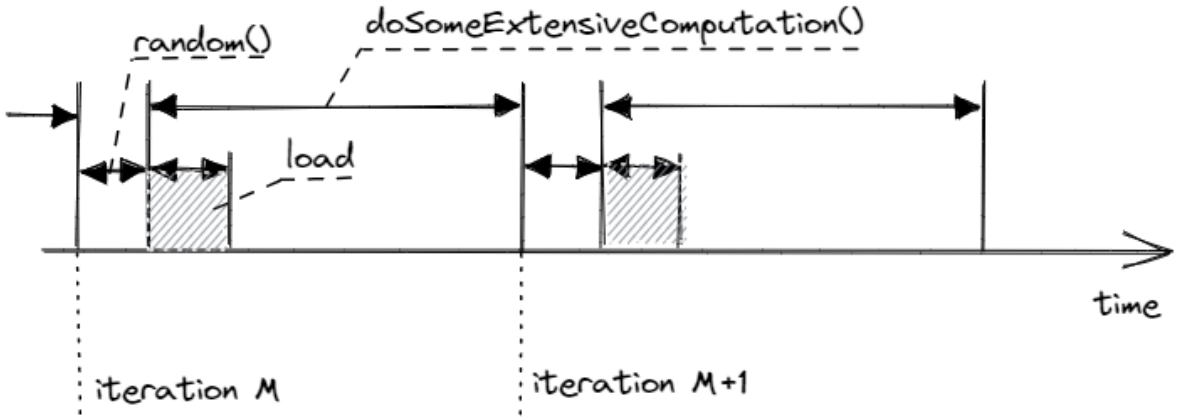



Figure 64: 通过与其他执行重叠来隐藏缓存未命中延迟.

预取的情况。Listing 148 中的代码就是这种情况之一，当图非常稀疏并且有很多顶点时，访问 `this->out_neighbors` 和 `this->in_neighbors` 向量很可能会错过缓存。

这段代码与前面的例子不同，因为每个迭代中都没有大量计算，所以缓存未命中的惩罚很可能主导了每个迭代的延迟。但是我们可以利用我们知道未来将访问的所有元素这一事实。向量 `edges` 的元素被顺序访问，因此很可能由硬件预取器及时地引入 L1 缓存。我们在这里的目标是将缓存未命中延迟与执行足够多的迭代重叠，以完全隐藏它。

一般来说，为了让预取提示有效，它们必须提前插入，以便在加载的值用于其他计算时，它已经存在于缓存中。但是，也不应该插入得太早，因为它可能会污染缓存，使数据长时间未使用。请注意，在 Listing 148 中，`lookAhead` 是一个模板参数，它允许尝试不同的值并查看哪个值能提供最佳性能。更高级的用户可以尝试使用 Section 6.4.4 中描述的方法估计预取窗口，在 easyperf 博客上可以找到使用这种方法的例子。[148]

代码清单: 接下来 8 次迭代的 SW 预获取示例。

```
template <int lookAhead = 8>
void Graph::update(const std::vector<Edge>& edges) {
  for(int i = 0; i + lookAhead < edges.size(); i++) {
    VertexID v = edges[i].from;
    VertexID u = edges[i].to;
    this->out_neighbors[u].push_back(v);
    this->in_neighbors[v].push_back(u);

    // prefetch elements for future iterations
    VertexID v_next = edges[i + lookAhead].from;
    VertexID u_next = edges[i + lookAhead].to;
    __builtin_prefetch(this->out_neighbors.data() + v_next);
    __builtin_prefetch(this->in_neighbors.data()  + u_next);
  }
  // process the remainder of the vector `edges` ...
}
```

软件内存预取最常用于循环中，但也可以将这些提示插入到父函数中，这再次取决于可用的预取窗口。

[148] "精确计时的机器代码与 Linux perf"- https://easyperf.net/blog/2019/04/03/Precise-timing-of-machine-code-with-Linux-perf#application-estimating-prefetch-window。

这种技术是一个强大的武器，但应该非常谨慎地使用，因为它并不容易正确使用。首先，显式内存预取不可移植，这意味着如果它在一个平台上带来了性能提升，并不保证在另一个平台上也能获得类似的加速。它非常依赖于实现，并且平台不需要遵守这些提示。在这种情况下，它可能会降低性能。我的建议是使用所有可用工具验证影响是积极的。不仅要检查性能数字，还要确保缓存未命中数量（尤其是 L3）下降。一旦将更改提交到代码库中，请监控您运行应用程序的所有平台上的性能，因为它可能对周围代码的更改非常敏感。如果收益不超过潜在的维护负担，请考虑放弃这个想法。

对于一些复杂的场景，请确保代码实际预取了正确的内存位置。当循环的当前迭代依赖于前一个迭代时，事情可能会变得棘手，例如存在 `continue` 语句或通过 `if` 条件改变要处理的下一个元素。在这种情况下，我的建议是使用工具代码来测试预取提示的准确性。因为使用不当，它会通过驱逐其他有用数据来降低缓存的性能。

最后，显式预取会增加代码大小，并增加 CPU 前端压力。预取提示只是一个进入内存子系统的伪加载，但没有目标寄存器。就像任何其他指令一样，它会消耗 CPU 资源。请极其谨慎地应用它，因为使用错误时，它可能会降低程序的性能。

## 8.3 内存分析

在软件开发和性能优化的过程中，内存分析是一个关键的步骤。它帮助开发者理解程序如何使用内存，以及如何优化内存使用以提高性能。以下是一些关于内存分析的要点：

1. **测量内存占用**：这涉及到使用工具来跟踪程序在运行时的内存使用情况。这可以帮助识别内存泄漏（程序结束时未释放的内存）和内存使用的峰值。在 Linux 系统中，常用的工具包括`valgrind`的`massif`工具，`gperftools`的`heap profiler`等。
2. **可视化内存访问热图**：内存访问热图是一种可视化工具，它显示了程序中不同部分的内存访问模式。热点（热图中的热点区域）表示频繁访问的内存区域，这些区域可能是性能优化的目标。例如，如果一个数据结构被频繁访问，那么将其放在快速的缓存中可能会提高性能。
3. **性能方法**：内存分析是性能分析的一部分。性能方法通常包括一系列的工具和技术，用于测量和优化程序的执行速度。内存分析可以提供关于程序性能瓶颈的重要信息，例如，如果程序的内存访问模式导致缓存未命中，那么可能需要重新设计数据结构或算法来改善缓存利用率。

见：easyperf Memory Profiling 博客文章[149], 翻译成中文的博客文章[150]。

## 8.4 减少 DTLB 未命中

正如本书前面所述，TLB 是一个快速但有限的每个内核缓存，用于将内存地址的虚拟到物理地址转换。如果没有它，应用程序每次内存访问都需要耗时的内核页表遍历来计算每个引用虚拟地址的正确物理地址。在具有 5 级页表的系统中，它将需要访问至少 5 个不同的内存位置才能获得地址转换。在 Section 11.8 部分，我们将讨论如何将大页面用于代码。在这里，我们将看到它们如何用于数据。

任何随机访问大内存区域的算法都可能遭受 DTLB 未命中之苦。这类应用程序的例子包括：在大数组中进行二进制搜索，访问大型哈希表，遍历图。使用大页面有可能加速这类应用程序。

在 x86 平台上，默认页面大小为 4KB。考虑一个应用程序主动引用数百 MB 的内存。首先，它需要分配许多小页面，这代价很高。其次，它将触及许多 4KB 大小的页面，每个页面都将在有限的一组 TLB 条目中竞争。例如，使用 2MB 的大页面，可以使用仅仅十个页面映射 20MB 的内存，而使用 4KB 的页面，您将需要 5120 个页面。这意味着需要的 TLB 条目更少，从而减少了 TLB 未命中次数。由于 2MB 条目的数量少得多，因此不会按 512 的比例减少。例如，在英特尔的 Skylake 内核系列中，L1 DTLB 为 4KB 页面提供 64 个条目，为 2MB 页面仅提供 32 个条目。除了 2MB

[149] easyperf-Memory-Profiling
[150] 译：内存分析

的大页面，AMD 和英特尔的 x86 架构芯片还支持 1GB 的超大页面，这些页面仅可用于数据，不能用于指令。使用 1GB 页面而不是 2MB 页面可以进一步减少 TLB 压力。

使用大页面通常会导致更少的页面遍历，并且在 TLB 未命中情况下遍历内核页表的惩罚也会减少，因为表本身更加紧凑。利用大页面的性能提升有时可以高达 30%，具体取决于应用程序遇到的 TLB 压力有多大。期望 2 倍的加速会要求太高，因为 TLB 未命中是主要瓶颈的情况相当罕见。论文 [Luo et al., 2015] 介绍了在 SPEC2006 benchmark 套件上使用大页面的评估。结果可以总结如下。在套件中的 29 个基准测试中，有 15 个的加速在 1% 以内，可以忽略不计。六个基准测试的加速范围在 1%-4% 之间。四个基准测试的加速范围在 4% 到 8% 之间。两个基准测试的加速分别为 10%，而获得最大收益的两个基准测试分别享受了 22% 和 27% 的加速。

许多现实世界的应用程序已经利用了大页面，例如 KVM、MySQL、PostgreSQL、Java JVM 等。通常，这些软件包提供了一个启用该功能的选项。每当您使用类似应用程序时，请查看其文档，了解是否可以启用大页面。

Windows 和 Linux 都允许应用程序建立大页面内存区域。有关如何在 Windows 和 Linux 上启用大页面的说明，请参见附录 C。在 Linux 上，应用程序中有两种使用大页面的方式：显式大页面和透明大页面。Windows 的支持不像 Linux 那么丰富，将在以后讨论。

### 8.4.1 显式大页面 (EHP)。

显式大页面 (EHP) 是系统内存的一部分，作为大页面文件系统 `hugetlbfs` 暴露。顾名思义，EHP 应在启动时或运行时预留。有关如何操作的说明，请参见附录 C。在启动时预留 EHP 可以增加分配成功的可能性，因为内存尚未严重碎片化。显式预分配的页面驻留在预留的内存块中，并且在内存压力下无法换出。此外，该内存空间无法用于其他目的，因此用户应谨慎分配，仅预留他们需要的页面数量。

在应用程序中使用 EHP 的最简单方法是在 `mmap` 中调用 `MAP_HUGETLB`，如 Listing 8.4.1 所示。在此代码中，指针 `ptr` 将指向一个 2MB 的内存区域，该区域是显式预留给 EHP 的。请注意，由于 EHP 没有预先保留，分配可能会失败。应用程序中使用 EHP 的另一种不太流行的方法可以在附录 C 中找到。此外，开发人员可以编写自己的基于 arena 的分配器，利用 EHP 进行分配。

代码清单: 从显式分配的巨大页面映射内存区域。

```
void ptr = mmap(nullptr, size, PROT_READ | PROT_WRITE,
                MAP_PRIVATE | MAP_ANONYMOUS | MAP_HUGETLB, -1, 0);
if (ptr == MAP_FAILED)
  throw std::bad_alloc{};
...
munmap(ptr, size);
```

过去，可以使用 libhugetlbfs: https://github.com/libhugetlbfs/libhugetlbfs[151] 库，该库允许覆盖现有动态链接的可执行文件使用的 `malloc` 调用，从而在大页面 (EHP) 之上分配内存。不幸的是，此项目已不再维护。它不需要用户修改代码或重新链接二进制文件。他们只需用 `LD_PRELOAD=libhugetlbfs.so HUGETLB_MORECORE=yes <your app command line>` 预先填充命令行即可使用它。但幸运的是，还有其他库允许使用大页面（非 EHP）和 `malloc`，我们稍后会看到。

### 8.4.2 透明大页面 (THP)

Linux 还提供透明大页面支持 (THP)，它具有两种操作模式：系统范围和每个进程。启用系统范围的 THP 时，内核会自动管理大页面，这对应用程序是透明的。操作系统内核尝试在需要大量内存块时将大页面分配给任何进程，并

[151] libhugetlbfs - https://github.com/libhugetlbfs/libhugetlbfs.

且如果可以分配这样的页面，则无需手动保留大页面。如果启用每个进程的 THP，内核仅将大页面分配给单个进程的内存区域，这些区域归因于 madvise 系统调用。您可以使用以下命令检查系统中是否启用了 THP：

```
$ cat /sys/kernel/mm/transparent_hugepage/enabled
always [madvise] never
```

如果值为 always（系统范围）或 madvise（每个进程），则 THP 可供您的应用程序使用。有关每个选项的详细规范，请参见 Linux 内核文档 [152] 中关于 THP 的内容。

启用系统范围的 THP 时，将自动将大页面用于常规内存分配，而无需应用程序明确请求。基本上，要观察大页面对应用程序的影响，用户只需启用系统范围的 THP，使用 echo "always" | sudo tee /sys/kernel/mm/transparent_hugepage/enabled 即可。它将自动启动名为 khugepaged 的守护进程，该进程开始扫描应用程序的内存空间，将普通页面提升为大页面。不过，有时内核可能无法将普通页面提升为大页面，因为找不到 2MB 的连续内存块。

系统范围的 THP 模式适用于快速实验，以检查大页面是否能提高性能。它可以自动工作，即使对于不知道 THP 的应用程序也是如此，因此开发人员不必更改代码即可看到大页面对他们应用程序的好处。

启用系统范围的大页面时，应用程序最终可能会分配更多的内存资源。应用程序可能会映射一个大区域，但只触碰其中 1 个字节，在这种情况下，可能会分配一个 2MB 的页面而不是 4k 的页面，这毫无意义。这就是为什么可以禁用系统范围的 THP，只让它们存在于 MADV_HUGEPAGE madvise 区域中，我们将在下面讨论这一点。完成实验后，请记得禁用系统范围的 THP，因为它可能不会使系统上运行的每个应用程序都受益。

使用 madvise（每个进程）选项时，THP 仅在通过 madvise 系统调用并带有 MADV_HUGEPAGE 标志的内存区域内启用。如 Listing 152 所示，指针 ptr 将指向一个 2MB 的匿名（透明）内存区域，该区域由内核动态分配。如果内核找不到 2MB 的连续内存块，mmap 调用可能会失败。

代码清单: 将内存区域映射到一个透明的巨大页面。

```
void ptr = mmap(nullptr, size, PROT_READ | PROT_WRITE | PROT_EXEC,
                MAP_PRIVATE | MAP_ANONYMOUS, -1 , 0);
if (ptr == MAP_FAILED)
  throw std::bad_alloc{};
madvise(ptr, size, MADV_HUGEPAGE);
// use the memory region `ptr`
munmap(ptr, size);
```

开发人员可以根据 Listing 152 中的代码构建自定义 THP 分配器。但是，他们还可以将 THP 用于应用程序进行的 malloc 调用中。许多内存分配库通过覆盖 libc 的 malloc 实现来提供此功能。以下是最流行的此类库之一 jemalloc 的示例。

如果您拥有应用程序的源代码，则可以使用附加的 -ljemalloc 选项重新链接二进制文件。这将使您的应用程序与 jemalloc 库动态链接，该库将处理所有 malloc 调用。然后使用以下选项启用堆分配的 THP：

```
$ MALLOC_CONF="thp:always" <your app command line>
```

如果您没有源代码，仍然可以通过预加载动态库来利用 jemalloc：

```
$ LD_PRELOAD=/usr/local/libjemalloc.so.2 MALLOC_CONF="thp:always" <your app command line>
```

Windows 只提供类似于 Linux THP 每个进程模式的方式使用大页面，通过 WinAPI VirtualAlloc 系统调用。有关详细信息，请参见附录 C。

[152] Linux kernel THP documentation - https://www.kernel.org/doc/Documentation/vm/transhuge.txt

### 8.4.3 显式大页面 (EHP) vs. 透明大页面 (THP)

Linux 用户可以使用三种不同的模式使用大页面：

- 显式大页面 (EHP)
- 系统范围透明大页面 (THP)
- 每个进程透明大页面 (THP)

让我们比较一下这些选项。首先，EHP 预先保留在虚拟内存中，而 THP 则没有。这使得使用 EHP 的软件包更难以交付，因为它们依赖于机器管理员所做的特定配置设置。此外，EHP 静态驻留在内存中，占用宝贵的 DRAM 空间，即使它们未使用。

其次，系统范围的透明大页面非常适合快速实验。无需更改用户代码即可测试在您的应用程序中使用大页面的好处。但是，将软件包运送给客户并要求他们启用系统范围的 THP 是不明智的，因为这可能会对该系统上运行的其他程序产生负面影响。通常，开发人员会识别代码中可以从大页面中受益的分配，并在这些位置使用 `madvise` 提示（每个进程模式）。

每个进程的 THP 没有上述任何一个缺点，但它还有一个缺点。之前我们讨论过，内核分配 THP 对用户来说是透明的。分配过程可能涉及多个内核进程，这些进程负责在虚拟内存中腾出空间，这可能包括将内存换出到磁盘、碎片化或提升页面。透明大页面的后台维护会产生内核在管理不可避免的碎片和交换问题时产生的非确定性延迟开销。EHP 不受内存碎片化影响，也不能换出到磁盘，因此延迟开销要小得多。

总而言之，THP 更易于使用，但会产生更大的分配延迟开销。这正是 THP 在高频交易和其他超低延迟行业不受欢迎的原因，他们更喜欢使用 EHP。另一方面，虚拟机提供商和数据库往往倾向于使用每个进程的 THP，因为要求额外的系统配置会给他们的用户带来负担。

## 8.5 问题与练习

1. 完成 `perf-ninja::data_packing` 实验作业，其中你需要使数据结构更紧凑。
2. 通过为未来循环迭代实现显式内存预取，解决 `perf-ninja::swmem_prefetch_1` 实验作业。
3. 使用我们在 Section 8.4 中讨论的方法解决 `perf-ninja::huge_pages_1` 实验作业。观察性能、`/proc/meminfo` 中的大页面分配以及测量 DTLB 加载和未命中次数的 CPU 性能计数器的任何变化。
4. 描述一段代码成为缓存友好的需要哪些条件?
5. 运行您每天使用的应用程序。测量其内存占用情况，分析并识别热点内存访问。它们是缓存友好的吗? 有没有办法改进它们?

**AI 回答：**

这些问题似乎是关于性能优化的实验或练习。以下是针对每个问题的建议性回答：

1. **perf-ninja::data_packing 实验作业**：要完成这个作业，你需要对现有的数据结构进行优化，使其在内存中占用更少的空间。这通常涉及到减少结构体中的填充（padding），使用位域（bit fields）来表示小的数据类型，以及重新排列结构体成员以提高内存对齐和缓存效率。例如，如果你有一个包含多个小整数的结构体，可以考虑将它们打包到一个更大的整数类型中，以减少内存占用。

2. **perf-ninja::swmem_prefetch_1 实验作业**：在这个作业中，你需要在代码中添加预取指令，以便在实际需要数据之前将其加载到缓存中。这可以通过在循环中适当位置插入预取指令（如 `__builtin_prefetch`）来实现。预取指令应该在数据被实际访问之前足够早地发出，以便在数据到达缓存时，CPU 已经准备好处理它。

3. **perf-ninja::huge_pages_1 实验作业**：为了解决这个作业，你需要配置系统以使用大页面（Huge Pages），并观察这对应用程序性能的影响。在 Linux 系统中，你可以通过修改系统配置或使用特定的系统调用来启用大页面。

然后，你需要监控应用程序的性能指标，如 CPU 缓存未命中次数，以及通过 `/proc/meminfo` 查看大页面的分配情况。这可以帮助你理解大页面对内存访问模式的影响。

4. **缓存友好的代码条件**：要使代码缓存友好，需要满足以下条件：
   - 数据访问模式应该遵循时间局部性和空间局部性原则。
   - 数据结构应该紧凑且对齐，以减少缓存行的浪费。
   - 避免频繁的内存分配和释放，特别是在循环中。
   - 使用适当的数据布局和对齐来最大化缓存利用率。
   - 在可能的情况下，使用大页面来减少 TLB 未命中。
5. **测量应用程序的内存占用和热点内存访问**：要测量应用程序的内存占用，你可以使用各种工具，如 Valgrind、gperftools、或者使用操作系统提供的工具，如 Linux 的 `top`, `htop`, 或 `vmstat`。要分析热点内存访问，你需要使用性能分析工具（如 Linux 的`perf`工具）来识别哪些数据结构或函数导致了频繁的内存访问。如果发现某些部分不是缓存友好的，你可以尝试优化数据结构，例如通过减少结构体大小、使用缓存对齐或者调整数据访问模式。在实际应用中，这可能需要对代码进行深入的分析和重构。

## 8.6 章节总结

- 在现实世界的应用中，大多数性能瓶颈都与 CPU 后端有关。这并不令人惊讶，因为所有与内存相关的问题，以及低效的计算，都属于这一类。
- 内存子系统的性能增长并没有跟上 CPU 性能的增长速度。然而，在许多应用中，内存访问是性能问题的常见来源。加速这类程序需要重新审视它们的内存访问方式。
- 在 Chapter 8部分，我们讨论了一些流行的技巧，用于创建对缓存友好的数据结构、内存预取，以及利用大内存页来提高 DTLB（直接翻译表项缓冲）性能。